

• DDice •

A Chronicle of Commands for the Tabletop Herald

— Contents —

Dice Rolling	2
HP & Healing (Player)	4
Game Master Commands	5
Character & Profile	6
Config & Maintenance	10
NPC System	14
Fight System	17
Merits & Ranks	22
Activities	23
Quest Board	27

Dice Rolling

Type dice notation directly or use any prefix style (r, !r, !roll):

<code>1d20</code>	<i>(bare notation — no prefix needed)</i>
<code>r1d20+5</code>	
<code>r1d20+5 atk</code>	<i>(label shows in bold above result)</i>
<code>r1d20+5 <i>atk</i></code> <code>[flavour text on next line]</code>	<i>(supports <i>italic</i> and bold)</i>

Advantage & Disadvantage

<code>ra1d20+5</code>	<i>(rolls twice, takes higher)</i>
<code>rd1d20+5</code>	<i>(rolls twice, takes lower)</i>

Rerolls (Player)

<code>rr</code>	<i>(costs 1 token)</i>
<code>rra</code>	<i>(reroll with advantage)</i>
<code>rrd</code>	<i>(reroll with disadvantage)</i>

Shorthand: append a reroll set to a stat — **strrr**, **dexrra** (adv), **conrrd** (dis) — to reroll your last roll. A label may follow: **strrr atk**.

Guided Roll (/roll)

<code>/roll stat:Strength</code>	<i>(pick a stat from the dropdown)</i>
<code>/roll stat:Wisdom mode:Advantage</code>	<i>(advantage / disadvantage)</i>
<code>/roll dice:2d6+3 label:damage</code>	<i>(custom notation instead of a stat)</i>
<code>/roll stat:Dexterity success_check:true</code> <code>label:sneak flavour:*slips into shadow*</code>	<i>(label + RP text)</i>

Every part of **/roll** is a dropdown or field — stat, advantage, success check, label and RP flavour — so nothing has to be memorised. Choose either a **stat** (rolls 1d20 + that stat from your sheet) or custom **dice**. The result uses the same card as a typed roll.

Stat Quick Rolls

str con dex wis lck

(on their own — nothing else in the message)

strength constitution dexterity

(long names roll exactly the same)

wisdom luck

wisa dexd

(advantage / disadvantage · a label may follow)

r str atk

(a plain stat roll with a label)

?str atk

(the same, as a success check)

Rolls 1d20 + that stat from your saved character. Add **a** for advantage or **d** for disadvantage — **wisa**, **dexd**. Every stat answers to its **full name** as well as its short one, and they behave identically: **strength** is **str**, **wisdom** is **wisa**, **luck** is **lck**.

A stat roll **always shows your sheet** — stats, HP, rerolls and all — even with the profile embed switched off. A bare **1d20+4** tells nobody which stat it was or where the 4 came from, so the card is forced open where the numbers need explaining. Plain dice rolls still honour **/profile off**.

A plain stat name must be the whole message. **str**, **con**, **dex**, **wis** and **lck** are ordinary words in conversation, so anything typed after one means it is treated as chat and left alone — “Dex or strength can both be used to throw things” is a sentence, not a roll. To label a plain stat roll, put a prefix on it: **r str atk** or **?str atk**.

Forms that cannot be mistaken for words keep their labels as they always did — **wisa sneak**, **dexd guard**, **strrr atk**, **conrra hold the line**.

? Success Check Rolls

Prefix **?** instead of **r** to get a success outcome:

?1d20+5 ?ra1d20+5 ?rd1d20+5 ?rr ?str ?wisa

Result	Outcome
Natural 20 / max face	Critical Success (gold)
Natural 1 / min face	Critical Fail (red)
Total 15+	Success
Total 10-14	Partial Success
Total 2-9	Fail

Critical Hits & Fails (all rolls)

Natural max on any die → gold label + result.

Natural 1 on any die → red label + result.

HP & Healing (Player)

!hp +5

!hp -3

Heal (White Knight + WIS 5 only)

Targets another player — cannot self-heal.

!heal @user

!h @user

Roll Result	Effect	Charges Used
Natural 20	+2 HP restored	None
20+ (modified)	+2 HP restored	1
2 - 19	No heal	1
Natural 1	No heal	2

Rest Commands (Player)

Append **@user** to any command for Game Master targeting. Default amounts shown — Game Masters can change them with **/config rest**.

Command	HP	Rerolls	Heal Charges
lrest	Full	Full	Full
srest	Half	—	—
hpfull	Full	—	—
hphalf	Half	—	—

Short rest restores HP only by default. A Game Master can re-enable rerolls/heal or change any amount.

Game Master Commands

All commands in this chapter require the GM role.

Game Master Rolls

<code>gmr 1d20+5</code>	<i>(public roll in channel)</i>
<code>gmr 1d20+5 perception</code>	<i>(with label)</i>
<code>gmrs 1d20+5</code>	<i>(secret — sent to Game Master DMs only)</i>
<code>gmrs 1d20+5 stealth</code>	

Game Master HP & Rerolls Targeting

```
!hp @user +5      !hp @user -3
!hp +5 @user      !hp -3 @user
!rerolls @user +1  !rerolls @user -1
```

Game Master Rest Targeting

```
lrest @user  srest @user  hpfull @user  hphalf
@user
```

Preset Tags

```
/tag assign user:@player tag:Hero of Kalidale
/tag remove user:@player tag:Hero of Kalidale
/tag list user:@player
```

Custom Tags

```
/tag custom action:Create emoji:[any emoji]
name:MyTag

/tag custom action>Delete name:MyTag

/tag custom action:List
```

For server emojis: type `\:emojiname:` in chat to get the full ID.

Character & Profile

Start Here — Make Your Character

Build your character with /char create. One command sets the whole sheet at once — every stat, your order, your class and both weapons — so it arrives complete instead of in pieces.

<code>/char create str:14 con:12 dex:10 wis:5 lck:3 \</code>	<i>(the whole character, one</i>
<code>order:White Knight class:Hero \</code>	<i>command)</i>
<code>weapon1:Longsword weapon2:Tower Shield</code>	

Every option is optional — fill in what you have decided and run it again later to add the rest. The values for **order**, **class** and the weapon fields are listed further down this chapter.

This matters most where a GM has switched sheet approval on. /char create sends one request for the whole character; building the same sheet a field at a time sends a fresh request on every single change and buries the GMs in them. It matters again afterwards, because once your sheet is approved every field is locked and only a GM can change it — so /char create is your one clean run at the character you want.

The Point Budget

A player building their own character spends **exactly 15 points** across the five stats, and every stat needs **at least 1**. Not 14, not 16 — the full allowance, all of it placed. A Game Master can change both numbers for the server with **/config statallowance**.

<code>/char create str:5 con:4 dex:3 wis:2 lck:1</code>	<i>(15 exactly, nothing on zero — fine)</i>
<code>/char create str:4 con:4 dex:3 wis:2 lck:1</code>	<i>(14 — refused, 1 still to place)</i>
<code>/char create str:6 con:5 dex:3 wis:2 lck:1</code>	<i>(17 — refused, take 2 back off)</i>
<code>/char create str:9 con:5 dex:1 wis:0 lck:0</code>	<i>(over AND stats on 0 — both reported)</i>

The refusal is posted **in the channel you are working in**, not as a private note only you can see — so it sits alongside what you typed. A copy also goes to the **approval channel** with a jump link back to the attempt, so the Game Masters can see who is wrestling with the budget and step in without having to be in that channel. Nothing is submitted either way.

Break a rule and the sheet is refused on the spot with a note saying exactly what is wrong — both problems at once if you managed both — along with your current spread and running total, so you can see what to move. Nothing is saved and nothing goes to the GMs until it is legal. The same check runs on **/char set**, on a pasted sheet, on **/profile load** and on **/char submit**, so there is no way in around the back.

Adjusting one stat at a time with **/char set** only checks the ceiling, so you can shuffle points between stats freely — lowering one before raising another would be impossible otherwise. The sheet is saved either way, but it is held back from the Game Masters until the full allowance is placed, and it tells you how many points are still loose.

Game Masters are not limited by any of this — building a character for a player, adjusting one, or making their own. A Game Master can hand out whatever spread a story calls for, and NPCs are unaffected entirely.

Changing One Field Later

Use **/char set** to adjust a single thing after the fact — a stat that went up, a weapon you swapped. It is a touch-up tool, not the way to build a character from nothing.

/char set field:str value:14	<i>(one field at a time)</i>
/char set field:con value:12	<i>(HP auto-maxes to CON + 2)</i>
/char set field:lck value:3	<i>(rerolls auto-max to LCK)</i>
/char set field:str value:14 user:@player	<i>(Game Master only)</i>

Items & Pages

/char give user:@a item:A tarnished silver key \ note:Cold to the touch	<i>(Game Master only)</i>
/char take user:@a id:3	<i>(by the number in their inventory)</i>
/char summary user:@a	<i>(their sheet and every page at once)</i>
/char standing user:@a	<i>(merit and renown, and where each came from)</i>
/char rollhistory user:@a	<i>(every natural die they have rolled)</i>

Items are free text — whatever a story calls for. Activities can hand them over too, and a player reads them back with **/char inventory**. Lore submitted with **/char lore** arrives in the sheet approval channel with Approve / Reject buttons, and a rejection asks you why.

Knight Orders

These are the values to pass to **order:** on **/char create** — or to **/char set** if this is the only thing you are changing.

/char set field:order value:White Knight	<i>(also: Black, Gold, Grey, Blue,)</i>
/char set field:order value:Red Knight	<i>(Purple, Green, Red)</i>

Class, Weapons & Emojis

/char set field:class value:Hero	<i>(Hero / Vanguard / Defender / Siege Knight)</i>
/char set field:weapon1 value:Longsword	
/char weaponemoji slot:Weapon 1 emoji:[choose]	<i>(standard emoji dropdown)</i>
/char weaponemoji slot:Weapon 1 custom::sword:	<i>(paste a server emoji)</i>

Pick a standard emoji from the dropdown, or paste a server custom emoji in the “custom” field (it overrides the dropdown). The image export falls back to a sword glyph for custom emojis.

If a GM **rejects** your sheet you can fix it and try again yourself — change whatever they objected to with **/char set** or **/char create** and it goes straight back to them. If you think it

was right as it stood, **/char submit** sends it again unchanged. There is no limit on how many times a sheet can go back and forth.

Reading & Sharing Your Sheet

/char create ...

(see “Start Here” — builds the whole sheet)

/char show **/char show user:@player**

/char export **/char export format:Image**

/char submit

(resend your sheet after a rejection)

With sheet approval switched on, an export goes to the GMs first. Running **/char export** posts the sheet block into the approval channel with **Release to player / Decline** buttons; the player sees only a private note that it has been sent. When a GM releases it, the block arrives by DM — or back in the channel they exported from, if their DMs are closed. Exporting is not an edit: it never changes a sheet’s approval state and never stops anyone rolling. Exporting again replaces the previous request, so the channel holds one live entry per player. GMs export straight away, for themselves and for anyone else, and on servers not using approvals nothing changes.

Character Pages

Beyond the stat block, a character accumulates things worth keeping. Each page can be read for yourself or for someone else with **user:.**

/char summary

(the sheet and every page below, on one post)

/char inventory

(what they are carrying)

/char standing

(merit and renown, and where each came from)

/char rollhistory

(every natural die this character has rolled)

/char rollhistory sides:6

(the same for a different die)

/char lore

(write your lore and send it to the Game Masters)

/char showlore

(read approved lore)

Inventory holds roleplay items — things an activity handed over, or a Game Master gave out by hand with **/char give user:@a item:A tarnished silver key**. Each carries a number, so **/char take user:@a id:3** removes one. Giving and taking are Game Master-only.

Standing puts total merit and renown at the top, then lists every movement beneath it — the quest, activity or Game Master award that caused it, the amount, and how long ago. Merits are a lifetime tally that only climbs; renown is a currency that is spent again.

Roll history counts every natural die a character has ever rolled, drawn as a bar chart with the natural 20s and 1s marked. It follows the character, not the player, and covers every roll they make — typed, slash, in a fight or in an activity.

Lore opens a writing box, pre-filled with whatever you wrote last time. It goes to the same channel character sheets do, with Approve / Reject buttons; a rejection asks the Game Master why and passes the reason back. Only approved lore shows on **/char showlore** — before that it says it is waiting, or why it was turned down. Rewriting retires the old request and sends a fresh one.

Nothing here is required. A character works perfectly well with an empty inventory, no lore and no renown — these are for tables that want to track more.

Profile

/profile on	/p on	(enable embed, max HP + rerolls)
/profile off	/p off	(disable embed, plain text rolls)
/profile show	/p show	(preview card without rolling)
/profile save	mysave	(snapshot current state)
/profile load	mysave	(restore snapshot in any channel)

Sheet Import

Paste an exported sheet into any channel the bot watches.

[paste sheet into channel]	(player)
[paste sheet] @player	(Game Master importing for another player)

Config & Maintenance

Config (Admin only)

<code>/config gmrole role:@Role</code>	<i>(add a GM role — several can be set)</i>
<code>/config gmrole role:@Role remove:true</code>	<i>(remove one)</i>
<code>/config gmrole role:@Role replace:true</code>	<i>(make it the only GM role)</i>
<code>/config gmrole</code>	<i>(list current GM roles)</i>
<code>/config heal charges 3</code>	
<code>/config statallowance points:15 minimum:1</code>	<i>(player build points (omit both to view))</i>
<code>/config hpbases base:3</code>	<i>(max HP = CON + this (default 2))</i>
<code>/config autorestore action:List</code>	<i>(every recovery schedule)</i>
<code>/config autorestore action:Add or update name:Breather \</code> <code>hours:6 hp:50% rerolls:0% heal:0%</code>	<i>(a light top-up)</i>
<code>/config autorestore action:Run now name:Breather</code>	<i>(fire one immediately)</i>
<code>/config npcchannel #channel</code>	<i>(set the NPC image bank channel)</i>
<code>/config npcrollback threshold:8</code>	<i>(NPC auto-reroll on $\text{nat} \leq N$ — 0 disables)</i>
<code>/config fightping enabled:true</code>	<i>(@-mention players on their turn — off by default)</i>
<code>/config rollaudit channel:#gm-rolls</code>	<i>(mirror every roll — raw input, result and jump link)</i>
<code>/config rollaudit test:true</code>	<i>(send a test mirror, report problems)</i>
<code>/config npcstats enabled:true</code>	<i>(reveal NPC stat blocks — hidden by default)</i>
<code>/config approvals channel:#sheet-approvals</code>	<i>(new sheets need GM sign-off)</i>
<code>/config approvals list:true</code>	<i>(every sheet still waiting — from the database)</i>
<code>/config approvals disable:true</code>	<i>(turn approval off)</i>
<code>/config rest type:Short Rest hp:50% rerolls:0%</code>	<i>(% of max)</i>
<code>/config rest type:Short Rest hp:3 rerolls:1</code>	<i>(flat numbers)</i>
<code>/config cleanwebhooks</code>	<i>(remove orphaned NPC webhooks)</i>

NPC roll cards hide STR/CON/DEX/WIS/LCK, the roll modifier and HP totals by default. Players see the name, order, the final roll total, the exact damage each hit deals, and a condition (□ unhurt / wounded / badly hurt / near death / down) instead of numbers — so the fight reads clearly

without revealing what an NPC can take. Reveal everything with **/config npcstats enabled:true**. NPC management commands are Game Master-only regardless.

A sheet that is **still waiting** can be edited by its owner — spot a mistake before a Game Master gets to it and you can fix it, which retires the old request and posts a fresh one. Only an **approved** sheet is frozen to its owner.

With an approval channel set, a player's new sheet is posted there for sign-off, pinging every GM role, with Approve / Reject buttons. Until approved they can't roll, heal or take fight actions — and once approved, their whole sheet can only be changed by a GM. Sheets a GM creates or edits skip the queue, and sheets made before approval was enabled keep working until someone edits them.

Every way a player can write to their own sheet goes to the queue: **/char create**, **/char set** (any field — stats, order, class and weapons alike), **/char weaponemoji**, **/profile load** and pasting an exported sheet. Building a character one **/char set** at a time is not a way past a GM. Editing a sheet that is still pending retires the old request and posts a fresh one, so the channel holds one live entry per player rather than a pile of stale ones. A rejected sheet can still be edited by its owner — that is how they fix it — and each edit sends it straight back for another look.

Pressing **Reject** opens a box asking **why**. The note is optional, but it travels with the decision — it is written onto the request in the approval channel, sent to the player with the rejection, and shown again if they try to roll before fixing it, so nobody is left guessing at what to change. Declining an **export** asks the same way.

A **rejection is never the end of it**. A rejected sheet stays editable by its owner, so the player fixes whatever the GM objected to with **/char set** or **/char create** and rejoins the queue on their own. If they believe it was right as it stood, **/char submit** sends it again unchanged. There is no limit — a sheet can go back and forth as many times as it takes, and each resubmission retires the previous request so the channel still holds one live entry per player. The rejection notice spells out both routes, so nobody is left waiting on a GM to do it for them.

A declined **export** works the same way — running **/char export** again puts a fresh request in front of the GMs.

Sheets are accepted from **any channel the bot can read** — an ordinary text channel, a thread, a forum post, an announcement channel, or the text chat inside a **voice or stage channel**. Wherever it came from is recorded with the request, so the decision notice can find its way back there if the player's DMs are closed.

The queue lives in the database, not in a Discord message. **/config approvals list:true** shows every sheet still waiting — who, how long ago, and which channel it came from — even if the request was never posted, was deleted, or landed somewhere nobody reads. If the bot cannot post to the approval channel it says so on that list and pings the GM roles in the channel the sheet was submitted from, so a player is never left locked out in silence.

The roll-audit mirror records every roll, in every channel, with nothing skipped. Prefix rolls, stat shorthand, success checks, rerolls, heals, **/roll**, **/dr** and every fight roll — plus Game Master rolls, including secret **gmrs** ones, so Game Masters are accountable to one another.

A Game Master rolling as an NPC with **/pr roll** is logged under their own name, tagged with the NPC they spoke as — the roll itself goes out through the NPC's webhook, so the audit is the only place it ties back to a person.

Rolls the bot makes for itself are recorded too, attributed to the fighter and tagged **auto**: auto-pilot attacks, defences and reroll answers, initiative at the start of every fight and whenever an NPC joins mid-fight, every roll of a full **/fight auto**, and demo bouts. Rolls typed inside the audit channel itself are mirrored as well — a secret **gmrs** there goes to the Game Master's DMs, so without the mirror it would leave no record at all.

Use `/config rollaudit test:true` to check it works. Set the channel's Discord permissions so only Game Masters can view it.

Several GM roles can be set at once — holding any of them grants GM access. Anyone with the Discord **Manage Server** permission always counts as a GM, so you can never lock yourself out by mis-setting a role.

Rest amounts come in two shapes, and the difference matters. A bare value **sets** the resource: **50%** puts them on half their maximum, **4** puts them on exactly 4 — even if that is fewer than they had. Prefix a **+** and it **adds** instead: **+4** gives four more HP, **+25%** gives a quarter of their maximum on top of what they have. Both cap at the maximum and both round down. **0%** leaves that resource untouched, and only the values you provide change.

Scheduled Recovery

A server can run **any number of named schedules**, each with its own timing and its own strength. A light top-up every few hours and a full recovery overnight can sit side by side.

<code>/config autorest action:Add or update name:Breather \</code>	<i>(half HP, rounded down, every</i>
<code>hours:6 hp:50% rerolls:0% heal:0%</code>	<i>6h)</i>
<code>/config autorest action:Add or update name:Full</code>	<i>(everything back, once a day)</i>
<code>Recovery \</code>	
<code>hours:24 hp:100% rerolls:100% heal:100%</code>	
<code>/config autorest action:List</code>	<i>(what is set, and when each</i>
	<i>next falls)</i>
<code>/config autorest action:Run now name:Breather</code>	<i>(fire one immediately)</i>
<code>/config autorest action:Pause name:Breather</code>	<i>(stop it without deleting it)</i>
<code>/config autorest action:Remove name:Breather</code>	<i>(delete it)</i>

Amounts use the same tokens as `/config rest`. A bare value **sets** the resource — **100%** full, **50%** half, **4** exactly four. A **+** prefix **adds** instead: **+4** is four more HP, **+25%** a quarter of their maximum on top. **0%** leaves it alone. Everything caps at the maximum and rounds down, so half of 11 HP is 5. A typo is refused when you set the schedule rather than quietly doing nothing at three in the morning.

Anyone on a quest that is in progress is skipped by every schedule. Their HP, rerolls and charges stay exactly where they are until the quest is completed, so being out in the field costs something. Applicants, and members of quests still open or already finished, are restored as normal. Heal charges only go to White Knights with WIS 5+, as everywhere else.

Give a schedule a **channel:** and each run is announced there, naming what it did, who was restored and who was left out in the field.

Each schedule carries its own clock in the database, so a restart or redeploy can neither skip a cycle nor fire one early. Adding a schedule, or resuming a paused one, starts its count from that moment.

Help & Maintenance

<code>/help</code>	(overview of all command groups)
<code>/help category:dice</code>	(detail on a specific group)
<code>/lastroll</code>	(recall your last roll in this channel)
<code>/backup now</code>	(export the database to this channel — Game Master)
<code>/backup auto channel:#backups</code>	(daily automatic backups — Game Master)

Destructive actions (`/npc delete`, `/fight end`, `/quest delete`, `/weapon remove`) ask for Confirm / Cancel before running.

Derived Stats

Max HP is **CON plus a flat base**, 2 by default — so CON 10 gives 12 HP. A Game Master can change the base for the whole server with `/config hpbased base:3`, making it CON+3; set it to 0 for max HP equal to CON alone. Everyone's ceiling moves the moment it is changed, players and NPCs alike, though current HP is left where it is — run a rest or `hpfull @user` to top people up.

Stat	Formula	On change
Max HP	CON + base	HP always maxes
Max Rerolls	LCK	Rerolls always max
Heal tracker	White Knight + WIS $\geq$ 5	Appears / disappears automatically

NPC System

All commands in this chapter are Game Master-only.

NPC Management

<code>/npc create name:Cave Orc str:14 con:10 dex:4 wis:2 lck:1 order:Black Knight</code>	<i>(full stat block)</i>
<code>/npc create name:Mystery Figure</code>	<i>(name only — stats added later)</i>
<code>/npc create name:Mystery Figure str:14 con:10</code>	<i>(fill stats in over time)</i>
<code>/npc copy name:Goblin new_name:Goblin 2</code>	<i>(duplicate an NPC, fresh HP)</i>
<code>/npc show name:Goblin</code>	<i>(full stat block for one NPC)</i>
<code>/npc hero name:Goblin stat:str</code>	<i>(make an NPC a Hero with a signature stat)</i>
<code>/npc hero name:Goblin remove:true</code>	<i>(strip Hero status)</i>
<code>/npc list</code>	<i>(shows stats and current HP)</i>
<code>/npc list category:Bandits</code>	<i>(only one category's NPCs)</i>
<code>/npc delete name:Aldric Vane</code>	

NPC HP & Healing

<code>/npc hp name:Goblin value:3</code>	<i>(set exact HP)</i>
<code>/npc hp name:Goblin</code>	<i>(omit value — full heal)</i>
<code>/npc heal names:all</code>	<i>(fully heal every NPC)</i>
<code>/npc heal names:Goblin, Orc</code>	<i>(fully heal the listed NPCs)</i>

Restoring Resources (/gmheal)

<code>/gmheal user:@a</code>	<i>(full HP — the default)</i>
<code>/gmheal user:@a restore:Everything</code>	<i>(HP, rerolls and heal charges)</i>
<code>/gmheal user:@a amount:Half</code>	<i>(restore half of maximum)</i>
<code>/gmheal user:@a amount:Add value:3</code>	<i>(add 3, capped at max)</i>
<code>/gmheal user:@a amount:Exact value:1</code>	<i>(set to an exact figure)</i>
<code>/gmheal npc:Goblin</code>	<i>(one NPC)</i>
<code>/gmheal npc:all</code>	<i>(every NPC at once)</i>

One command for every restore. Works on a player or an NPC (or **all** NPCs), and HP changes sync straight into any active fight. NPCs only have HP; heal charges are skipped for anyone who isn't a White Knight with WIS 5+.

global: restores everyone at once instead of naming a target — **Players, NPCs, or Everyone** together. It takes the same **amount** and **restore** options as a single target, so `/gmheal global:Everyone amount:Half` puts the whole server on half HP, and `/gmheal global:Players restore:Everything` hands every character HP, rerolls and heal charges

back. Pick exactly one of **user**, **npc** or **global**.

Unlike scheduled recovery, a global heal does **not** skip players out on a quest. A Game Master typing this has decided to heal the room, and a silent exclusion mid-session would be a nasty surprise. Heal charges still only reach White Knights with WIS 5+, and every change syncs into any fight already running.

Stats are optional — an NPC can be registered by name (and given an avatar) with no stats at all, then statted up later. Re-running **/npc create** with the same name updates only the fields you supply and leaves the rest untouched.

NPC HP persists between fights. Knocked-down NPCs (0 HP or less) are left out of new fights until restored.

Speaking as an NPC

<code>/pr say name:Cave Orc speech:Halt! Who goes there?</code>	<i>(speech — in quote marks)</i>
<code>/pr say name:Cave Orc action:raises its axe</code>	<i>(action — italic emote)</i>
<code>/pr say name:Cave Orc action:raises its axe speech:Halt!</code>	<i>(both, stacked)</i>
<code>/pr say name:Cave Orc</code>	<i>(opens a writing box)</i>

Posts as the NPC through their webhook — their name and avatar, no dice rolled. Fill **action**, **speech**, or both: the action is italicised and the speech is wrapped in quote marks, stacked on separate lines. Leave both blank and a **writing box** opens with roomy multi-line fields — easier for longer roleplay. Use **raw** to post exactly as typed.

Rolling as an NPC (/pr shorthand)

```
/pr roll name:Cave Orc notation:1d20+8 label:strike
flavour:The orc lunges

/pr create name:Aldric Vane str:8 con:6 dex:10 wis:4
lck:2

/pr list
```

Posts via webhook — appears as the NPC with their name and avatar.

Setting NPC Avatars

1. Admin runs **/config npcchannel #channel** to set the image bank channel.
2. Game Master uploads an image to that channel with the NPC name as the message text.
3. Bot adds a checkmark reaction to confirm. 4. Re-upload with the same name to update.

Bot requires Manage Webhooks permission in the server.

Heroes & Signature Stats

Hero is a Game Master-granted class, not a player choice — players who try to set it are refused. Both player sheets and NPCs can be Heroes.

<code>/char set field:class value:Hero user:@a</code>	<i>(grant Hero to a player sheet)</i>
<code>/char signature user:@a stat:str</code>	<i>(designate their signature stat)</i>
<code>/npc hero name:Aldric stat:dex</code>	<i>(Hero NPC with a signature stat)</i>

A Hero may have one **signature stat** with **5 or more** points. Every roll using that stat — typed shorthand, **/roll**, and both attack and defence in fights, manual or automatic — is made with **advantage**, marked ☐ on the roll card. If the stat later drops below 5 the advantage simply stops applying until it is restored; an explicitly requested disadvantage still wins.

Fight System

Structured fights with initiative, turn order, stat-based rolls and damage tracking.

Commands

<code>/fight start players:@a @b npcs:Goblin, Orc</code>	<i>(any number of each)</i>
<code>/fight start ... manual:true</code>	<i>(skip roll, keep listed order)</i>
<code>/fight start ... practice:true</code>	<i>(friendly bout — fighters yield at 2 HP)</i>
<code>/fight addnpc npc:Goblin, Orc</code>	<i>(add NPC(s) mid-fight, Game Master)</i>
<code>/fight order sequence:@a, Goblin, @b</code>	<i>(reorder players + NPCs, Game Master)</i>
<code>/fight atk stat:str target:@user</code>	<i>(attack a player)</i>
<code>/fight atk stat:str target_npc:Orc</code>	<i>(attack an NPC)</i>
<code>/fight atk stat:str npc:Goblin target:@user</code>	<i>(Game Master attacks AS the NPC)</i>
<code>/fight def stat:dex</code>	<i>(defend)</i>
<code>/fight def stat:dex npc:Goblin</code>	<i>(Game Master defends AS the NPC)</i>
<code>/fight rr</code>	<i>(reroll last fight roll — 1 token)</i>
<code>/fight resolve</code>	<i>(resolve the exchange)</i>
<code>/fight status</code>	<i>(show turn order and HP)</i>
<code>/fight log</code>	<i>(re-post the last finished fight's recap)</i>
<code>/fight skip</code>	<i>(skip the current turn — fighter stays in, Game Master)</i>
<code>/fight hp value:3 target_npc:Orc</code>	<i>(set HP mid-fight, sheet synced, Game Master)</i>
<code>/fight kick target_npc:Orc</code>	<i>(remove a fighter, fight continues, Game Master)</i>
<code>/fight refill npcs:all</code>	<i>("all" or names — refill reroll tokens, Game Master)</i>
<code>/fight auto mode:Full ...</code>	<i>(bot resolves whole fight, Game Master)</i>
<code>/fight auto mode:Full teams:@a @b vs Goblin, Orc</code>	<i>(party-vs-monsters sides, Game Master)</i>
<code>/fight auto mode:NPCs only ...</code>	<i>(bot plays NPCs, Game Master)</i>
<code>/fight auto mode:Demo</code>	<i>(example showcase, Game Master)</i>
<code>/fight auto ... practice:true</code>	<i>(a bout in any auto mode, Game Master)</i>
<code>/fight forfeit</code>	<i>(concede — HP preserved)</i>
<code>/fight end</code>	<i>(Game Master only)</i>

How a Fight Runs

When a fight starts, the bot rolls DEX initiative (1d20 + DEX) for every fighter and orders the turn order from highest to lowest, ties broken by the raw d20. Add **manual:true** to skip the roll and keep your listed order, or use **/fight order** with a comma-separated sequence.

Knocked-down fighters: anyone at 0 HP or less is left out when a fight starts or when NPCs are added, with a warning naming them. Restore NPCs with **/npc heal** or **/npc hp**, players with rests or **hpfull @user**.

Practice bouts: add **practice:true** to **/fight start** or **/fight auto** and the fight becomes a friendly spar. Everything runs exactly as normal — initiative, rolls, damage, crits, carry-over effects, rerolls, recap — but the cut-off moves from 0 HP to **2 HP**. A fighter bows out the moment they reach 2, and damage never carries anyone below it, so nobody leaves the yard worse than winded. Sparring partners already at 2 HP or less are left out at the start, the same way knocked-down fighters are. The bout is announced on start, marked on **/fight status**, and noted again in **/fight log**, so a spar can never be mistaken for the real thing.

HP stays in sync: any mid-fight HP change — **!hp**, **!heal**, rests, **/npc hp** or the **/fight hp** command — is mirrored straight into the fight, so a heal is never overwritten by the next exchange. Anywhere a command takes NPC names, **category:Name** adds a whole category at once.

When a Fight Ends

However a fight finishes — a knockout, a forfeit, a kick, **/fight end** or an auto-resolve — a single public **result post** goes to the channel where everyone can read it. Nothing important is left in a private reply: the Game Master's confirmation for **/fight end** stays private, but the result itself is posted for the whole table.

The post names the **victor**, then lists every combatant's **final standing** — exact HP for players, a condition band for NPCs while their stats are hidden — followed by the recap.

The **recap** covers players and NPCs alike, in three parts. **Rolls:** how many attacks and defences each fighter made, their average total, and their best and worst natural dice. **Damage:** dealt and taken, natural 20s and 1s, and rerolls spent. **Blow by blow:** every exchange in order — both rolls, the natural die and the final total for each side, and whether it landed or was blocked, with natural 20s and 1s marked. Very long fights show the most recent exchanges and say how many earlier ones were trimmed; the roll and damage figures still cover the whole fight. Long recaps are split across messages so nothing is lost to Discord's length limit.

/fight log re-posts the last finished fight's recap in that channel at any time, blow-by-blow included.

NPCs as combatants: a Game Master lists NPCs in **npcs:** (comma-separated) on start or **/fight addnpc** later. On an NPC's turn the Game Master adds **npc:Name** to **/fight atk** or **/fight def**; attack an NPC with **target_npc:Name**.

Auto Modes

/fight auto mode:Full rolls attack, defend and damage for everyone and plays the fight through to a winner, saving HP to each combatant's sheet as it changes. Add **teams:@a @b vs Goblin, Orc** for proper sides — fighters only target the enemy team, and the last team standing wins. **mode:NPCs only** starts a normal fight where the bot takes the NPCs' turns automatically while players still use **/fight atk** and **/fight def**. **mode:Demo** runs a throwaway example.

Automatic rolls always use the fighter's highest of STR or DEX, for attack and defence alike (ties go to STR), and post the same full roll card as a manual roll. Each NPC's cards appear under that NPC's own name and avatar (via webhook).

NPC rerolls: in auto modes each NPC carries its own reroll tokens — LCK per fight. A token is spent only when the natural die shows **8 or less** (server-tunable via **/config npcreroll**; 0 disables): a defender about to be hit rerolls first, then a blocked attacker may answer — one each per exchange. **/fight status** shows remaining tokens, and **/fight refill** restores them mid-fight.

Damage

Situation	Damage
Attack total $\geq$ Defence total	1
Attacker natural 20	+1 bonus
Defender natural 1	+1 bonus
Attacker nat 20 + Defender nat 1	4 total
Defence total $>$ Attack total	0 (blocked)
Fighter reaches 0 HP	Knocked down, removed from turn order, HP goes negative

Players can reroll using **rr** / **rra** / **rrd** after their initial roll before **/fight resolve**.

Critical Effects (carry-over)

A natural 1 or natural 20 leaves a mark on the *next* roll, in both manual and auto fights:

Trigger	Effect on the next roll
Natural 1 on an attack	The attacker fumbles — their next defence is rolled as a flat d20 (no stat, no advantage).
Natural 20 on a defence	The defence turns the blow aside and the defender gains +2 on their next attack — unless that attack was itself a natural 20.

Each effect is consumed by the affected fighter's next matching roll and is announced when it lands ("presses the riposte", "defends on a flat d20"). Leaving a fight clears any pending effects.

Merits & Ranks

A milestone-style progression system. Merits are a lifetime tally a Game Master awards; ranks are named tiers with merit thresholds. The bot tracks progress and flags eligibility, but every promotion is decided by a Game Master.

Merits

<code>/merit view</code>	<i>(your merits, rank, and merits to next rank)</i>
<code>/merit view user:@player</code>	<i>(view another player)</i>
<code>/merit leaderboard</code>	<i>(top earners on the server)</i>
<code>/merit add user:@player amount:2</code>	<i>(award merits — Game Master)</i>
<code>/merit remove user:@player amount:1</code>	<i>(take merits away — Game Master)</i>
<code>/merit set user:@player amount:10</code>	<i>(set an exact total — Game Master)</i>
<code>/merit history user:@player</code>	<i>(a player's merit timeline)</i>
<code>/merit history</code>	<i>(recent server-wide merit activity)</i>

Ranks

<code>/rank list</code>	<i>(all ranks and their thresholds)</i>
<code>/rank add name:Knight threshold:5</code>	<i>(create or update a rank — Game Master)</i>
<code>/rank add name:Squire threshold:0 order:0</code>	<i>(order sets junior→senior — Game Master)</i>
<code>/rank promote user:@player rank:Knight</code>	<i>(set a player's rank — Game Master)</i>
<code>/rank eligible</code>	<i>(who has met a threshold but isn't promoted — Game Master)</i>
<code>/rank remove name:Squire</code>	<i>(delete a rank — Game Master)</i>

/merit view shows current merits and exactly how many more are needed for the next rank, e.g. “Knight · 7 merits · 8 to Paladin”. Every merit change is recorded — quest rewards show the quest name, manual changes show “by GM” — so **/merit history** answers “who earned what, and when”. Removing a rank doesn't change players who already hold its label.

Activities

An activity is a minigame you write for your server: the bot narrates, asks for rolls, branches on the results and loops until someone stops. Fishing, foraging, a gauntlet in the training yard — whatever you script. Only a Game Master can write one; whether players can **start** one is a setting.

Writing One

Paste a script into any channel the bot can read, starting with **[ACTIVITY] Name**. Re-pasting the same name replaces it. The whole script is checked before anything is saved.

[ACTIVITY] Fishing

TALLY renown

(a running total, paid out at the end)

SCENE find

SAY Find a spot to set up.

ROLL wis DC15

(a difficulty to beat)

PASS -> cast

FAIL ONE OF

(picks a different line each loop)

This spot does not look all too lucky...

They are not biting here today...

FAIL -> find

(loops back)

SCENE cast

ROLL str|dex|wis

(the roller picks the stat)

1-5 Small fry. -> fight_small

(ranges on the total)

16+ A monster! -> fight_extra

SCENE fight_big

GAUNTLET str|con 14 12 10

(three rolls, each harder to fail)

NAT20 It leaps aboard. -> caught

NAT1 Your line snaps. -> restring

PASS -> caught

FAIL -> find

SCENE caught

GAIN renown 3

(banked on arrival)

CHOICE

(buttons, no dice)

Carry on -> cast

Call it quits -> depot

SCENE depot

END TALLY

(pays the tally out as renown)

What Each Line Does

Line	Meaning
SCENE name	A step. The first one is where a run begins.
SAY ...	Narration. Runs over as many lines as you like.
AS Cave Orc	Speak this scene in an NPC's voice, through their webhook.
ROLL str	Ask for a roll. str dex wis lets the roller choose.
ROLL wis DC15	Beat 15 for PASS, else FAIL.
GAUNTLET str con 14 12 10	A run of rolls, each with its own DC. All must pass.
GAUNTLET 14:str 12:str con 10:dex	The same, but a different check at every step.
1-5 text -> scene	Branch on the roll total. 16+ is open-ended.
PASS / FAIL text -> scene	Branch on the outcome band.
BAND ONE OF	Indented lines below become random variants.
NAT20 / NAT1 text -> scene	Overrides everything else.
CHOICE	Buttons instead of dice. Options are label -> scene .
TALLY renown	Names a running total, declared once at the top.
GAIN renown 3	Adds to the tally when a player arrives at this scene.
END	Finishes. END TALLY pays out, merits:2 awards merits,
	rewards:a silver key is announced for you to hand out.

Without a **DC** or ranges, outcomes fall back to the same bands a ? check uses: **CRIT** a natural 20, **PASS** 15+, **PARTIAL** 10-14, **FAIL** under 10, **FUMBLE** a natural 1. You need not define all of them — a crit falls back to PASS, a fumble to FAIL.

*Validation refuses a branch pointing at a scene that does not exist, a duplicate scene name, a scene with no roll, choice or ending, a roll on something that is not a stat, a gauntlet longer than eight, and a **GAIN** with no **TALLY** — each with the reason, so a run can never dead-end.*

Running One

<code>/activity demo</code>	<i>(play the built-in fishing game (Game Master))</i>
<code>/activity run name:Fishing</code>	<i>(start it in this channel)</i>
<code>/activity list</code> <code>/activity show name:Fishing</code>	<i>(what exists, and read it back in full)</i>
<code>/activity stop</code>	<i>(abandon the run here)</i>
<code>/activity set name:X scene:find field:Roll value:dex</code>	<i>(tweak one line (Game Master))</i>
<code>/activity delete name:X</code>	<i>(remove it (Game Master, asks first))</i>
<code>/config activities players:true</code>	<i>(let players start them too)</i>

/activity demo plays a ready-made fishing game so you can see the whole system working before writing anything: a difficulty check that loops with a different excuse each time, four sizes of catch chosen by the roll, a gauntlet per size, natural 20s and 1s, and buttons to keep going or head back. It awards **nothing** — no renown, no merits, no items — so it is safe to play with.

Each scene posts with a button per stat it accepts. **Anyone in the channel can press one** — the roll uses their own sheet, honours a Hero's signature stat, and lands in the roll audit like any other. One run per channel at a time. Writing and deleting always need a Game Master; starting one is GM-only until **/config activities players:true**.

Renown

Renown is not a currency. It is a running tally of how a character **stands in the world** — what they have done, who has noticed, and how far their name carries. It is earned from quests, encounters and activities, and it is not meant to be traded away for goods. Someone with high renown is **known**; that is the whole of it.

A Game Master can adjust it either way when the story calls for it — a reputation can be damaged as well as built — but it is a record of standing rather than a purse.

<code>/renown view</code> <code>/renown view user:@player</code>	
<code>/renown leaderboard</code>	<i>(who is best known)</i>
<code>/renown history</code>	<i>(where a standing came from)</i>
<code>/renown gain user:@a amount:5 reason:Cleared the Sunken Vault</code>	<i>(Game Master)</i>
<code>/renown loss user:@a amount:3 reason:Disgraced at court</code>	<i>(Game Master)</i>
<code>/renown set user:@a amount:0</code>	<i>(Game Master)</i>

Every change is logged with its reason, so **/renown history** answers how a reputation was built and where it was lost.

Merit

Merit is the earned measure of service — awarded by a Game Master, accumulated across quests and activities, and the thing rank thresholds are set against. Unlike renown, **merit is tradeable**: it can be passed between players, and potentially to and from NPCs, as payment, tribute, a debt settled or a favour bought.

`/merit give user:@a amount:2 reason:A debt settled` *(offer some of your merit)*

`/merit trades` *(what is still waiting on a Game Master)*

No merit moves until a Game Master signs it off. An offer is held and posted to the sheet approval channel with Approve / Refuse buttons; a refusal asks the Game Master why and passes the reason back. Both parties are told when it lands, and it is announced in the channel where it was offered so the table sees the trade happen.

Your balance is checked twice — when you offer, and again when a Game Master approves. If you have spent the merit in between, the trade is voided rather than pushing anyone into the red.

Renown says who you are in the world. Merit is what you have earned and may hand on. A character can be widely known and hold no merit at all, or quietly hold a great deal.

Quest Board

GMs create quests with lore, objectives, details and rewards. Quests are posted as a message with an Apply button and also appear on the board. Players apply, a Game Master approves the party, and on completion merits are auto-awarded while other rewards are listed for the Game Master to hand out.

For Players

<code>/quest board</code>	(list quests (filter: open/active/completed/all))
<code>/quest show number:1</code>	(full details of one quest)
<code>/quest roster number:1</code>	(see applicants and the party)
<code>/quest apply number:1</code>	(apply to join — or tap Apply on the post)
<code>/quest withdraw number:1</code>	(leave or cancel your application)
<code>/quest log user:@player</code>	(completed quests a player was on)

For the Game Master

<code>/quest create name:Goblin Cave objectives:... merit_reward:2 party_size:4 hard_cap:true</code>	(create a quest)
<code>/quest post number:1 channel:#board</code>	(post as an embed with an Apply button)
<code>/quest approve number:1 user:@a force:true</code>	(approve an applicant · force past a hard cap)
<code>/quest kick number:1 user:@a</code>	(remove a member or applicant)
<code>/quest runchannel number:1 channel:#thread</code>	(set where it runs & rewards)
<code>/quest start number:1</code>	(lock the party, mark in progress)
<code>/quest complete number:1</code>	(finish — auto-award merits, list rewards)
<code>/quest delete number:1</code>	(remove a quest permanently)

Quests are auto-numbered for easy recall, e.g. **#001-Goblin Cave** (repeatable quests keep the name and get a fresh number each time). Party size is a hard cap or a suggestion — the Game Master chooses with **hard_cap**, and **force:true** on approve overrides a cap. Point a quest at a thread with **/quest runchannel** and its completion rewards are announced there.

Penned by the DDice Scriptorium · May your rolls be ever in your favour